

A PROJECT REPORT ON

# PEER-TO-PEER SOLAR ENERGY TRADING MARKETPLACE

*A MERN Stack Based Simulation of a Decentralized Local Energy Trading Platform*

Submitted in partial fulfilment of the requirements for the award of the degree of  
**MASTER OF COMPUTER APPLICATIONS (MCA)**

Submitted by

***Mrutyunjaya Beura***

Enrolment / Roll No.: 12516743

**DEPARTMENT OF COMPUTER APPLICATIONS**  
**Lovely Professional University , Punjab**  
2025-27

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, [Guide Name], for the invaluable guidance, continuous encouragement, and constructive feedback provided throughout the duration of this project. Their expertise and insights were instrumental in shaping the direction and quality of this work.

I am also thankful to the Head of the Department of Computer Applications and all the faculty members of [College/Institute Name] for providing the necessary resources, infrastructure, and a conducive academic environment for carrying out this project successfully.

I extend my heartfelt thanks to my classmates and friends for their support, suggestions, and motivation during the various stages of design, development, and testing of this project. Their willingness to discuss ideas and review my work helped me refine several aspects of the system.

Finally, I am deeply grateful to my family for their unwavering support, patience, and encouragement throughout my academic journey. This project would not have been possible without the collective support of everyone mentioned above, and I remain sincerely thankful to all of them.

**Mrutyunjaya Beura**

## ABSTRACT

The **Peer-to-Peer Solar Energy Trading Marketplace** is a MERN Stack web application that simulates a local energy marketplace where solar energy prosumers can sell surplus electricity directly to nearby consumers. The system provides an alternative to the traditional grid model by enabling transparent and efficient energy trading between users.

The application is built using **MongoDB, Express.js, React.js, Node.js, Vite, and Tailwind CSS**, with **JWT authentication** for secure role-based access. It supports three user roles: **Prosumer, Consumer, and Grid Administrator**. Smart meter data is simulated to calculate energy generation, consumption, and surplus energy, which can be listed on the marketplace. A matching engine connects buyers with suitable sellers, while all transactions are securely recorded. Interactive dashboards provide real-time insights into energy production, consumption, transactions, and overall grid performance.

This project demonstrates the feasibility of a decentralized renewable energy trading platform using modern web technologies. Although designed as an academic simulation, the architecture can be extended with IoT-enabled smart meters, blockchain-based transactions, and AI-driven energy demand forecasting for real-world deployment.

**Keywords:** Peer-to-Peer Energy Trading, Solar Energy, Smart Grid, MERN Stack, Smart Meter Simulation, Renewable Energy Marketplace.

## 2. PROBLEM STATEMENT

Rooftop solar prosumers currently have very limited options for monetising their surplus energy beyond the fixed, utility-determined feed-in tariff. This creates several interconnected problems that this project seeks to address through a simulated but architecturally realistic software solution:

First, there is a lack of price transparency and negotiation power for prosumers, who must accept whatever buy-back rate is offered by the utility, regardless of the actual local supply-demand balance. Second, nearby consumers have no visibility into, or access to, the surplus renewable energy being generated in their immediate vicinity, even though purchasing it directly could be cheaper and more sustainable than drawing entirely from the centralized grid. Third, existing utility billing systems typically settle accounts on a monthly cycle, offering no real-time insight into how much energy is being generated, consumed, surplus, or earned/spent at any given moment. Fourth, there is no unified digital platform where all stakeholders — prosumers, consumers, and grid administrators — can view a consistent, real-time picture of local energy production, consumption, and trading activity. Finally, existing solutions rarely provide a role-based, secure, and auditable digital ledger of peer-to-peer energy transactions that could later be integrated with real smart-metering hardware or distributed-ledger settlement technology.

The core problem, therefore, is: how can a software platform be designed and implemented that realistically simulates smart-meter-driven surplus calculation, allows solar prosumers to list and sell that surplus directly to nearby consumers at negotiated or matched prices, and gives a grid administrator the tools to monitor the resulting marketplace and overall grid load, all while maintaining secure, role-based access and a reliable transaction record?

## 5. OBJECTIVES

The specific objectives of this project are as follows:

- To design and implement a secure, role-based authentication and authorization system supporting Prosumer, Consumer, and Grid Administrator roles using JSON Web Tokens (JWT).
- To build a smart meter simulation module capable of generating realistic, time-varying solar generation and consumption data for each user account.
- To implement automatic surplus energy computation based on simulated generation and consumption readings.
- To develop a marketplace module through which Prosumers can list surplus energy for sale, specifying quantity and price.
- To design and implement a buyer-seller matching engine capable of automatically pairing consumer purchase requests with one or more suitable seller listings.
- To maintain an accurate, auditable, and immutable transaction ledger recording every completed energy trade.
- To provide real-time, role-specific analytical dashboards using Recharts, showing generation, consumption, earnings, spending, and grid load trends.
- To provide the Grid Administrator with system-wide monitoring tools, including user management, transaction oversight, and marketplace analytics.
- To document the complete requirement analysis, system architecture, database design, and REST API specification of the system in a manner suitable for future extension.
- To validate the correctness and reliability of the system through structured unit and integration testing.

## 6. SCOPE OF THE PROJECT

The scope of this project is limited to a software simulation of a peer-to-peer solar energy trading marketplace intended for academic demonstration and learning purposes. It covers the complete design and implementation of a three-tier MERN stack web application including user registration and authentication, smart-meter data simulation, surplus energy computation, energy listing management, an automated matching engine, a transaction ledger, and role-specific real-time dashboards.

The project scope includes the frontend user interface for all three roles, the backend REST API layer with full CRUD operations for users, meter readings, listings, and transactions, and the MongoDB database schema design supporting these features. It also includes basic security measures such as password hashing, JWT-based session management, input validation, and role-based route protection, as well as a testing strategy comprising unit and integration tests for critical modules.

The scope explicitly excludes integration with real, physical smart-meter hardware or utility SCADA systems; real monetary payment gateway integration (all "earnings" and "purchases" are simulated using an internal virtual balance/ledger rather than real currency transactions); blockchain-based settlement or smart contracts, although these are discussed as potential future enhancements; and regulatory or legal compliance work required for an actual commercial energy-trading licence, which lies outside the domain of a software engineering academic project. The system is designed to be architecturally extensible so that these excluded capabilities could be added in a future phase without a fundamental redesign.

Finally, literature on smart-meter data and simulation highlights those typical household solar generation profiles follow a bell-shaped curve peaking around midday, while household consumption profiles typically show peaks in the morning and evening. This project's smart-meter simulation module draws on these established generation and consumption profile shapes to produce realistic, time-varying synthetic readings rather than uniformly random values, thereby providing a more meaningful basis for surplus computation and dashboard visualisation.

## 14. FRONTEND FOLDER STRUCTURE

The frontend is organized as a modular Vite + React application as shown below.

```

frontend/
├── index.html
├── package.json
├── vite.config.js           # Dev server + /api proxy
├── tailwind.config.js
├── postcss.config.js
├── src/
│   ├── main.jsx             # React entry
│   ├── App.jsx              # Routes
│   ├── index.css            # Global styles (Tailwind)
│   ├── api/
│   │   └── axios.js         # API client (baseUrl /api)
│   ├── context/
│   │   └── AuthContext.jsx  # Login / register / session
│   ├── components/
│   │   ├── DashboardLayout.jsx
│   │   ├── Sidebar.jsx
│   │   ├── StatCard.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── landing/        # Landing page sections
│   │       ├── Header.jsx
│   │       ├── Hero.jsx
│   │       ├── Features.jsx
│   │       ├── HowItWorks.jsx
│   │       ├── StatsBanner.jsx
│   │       ├── Testimonials.jsx
│   │       ├── FAQ.jsx
│   │       └── Footer.jsx
│   └── pages/
│       ├── Landing.jsx
│       ├── auth/
│       │   ├── Login.jsx
│       │   └── Register.jsx
│       ├── prosumer/
│       │   ├── ProsumerDashboard.jsx
│       │   └── ListingsPage.jsx
│       ├── consumer/
│       │   └── ConsumerDashboard.jsx
│       ├── admin/
│       │   ├── AdminDashboard.jsx
│       │   ├── AdminUsers.jsx
│       │   ├── AdminListings.jsx
│       │   ├── AdminTransactions.jsx
│       │   └── AdminMeters.jsx
│       └── shared/
│           ├── Marketplace.jsx
│           ├── WalletPage.jsx
│           └── TransactionsPage.jsx

```

## 15. BACKEND FOLDER STRUCTURE

The backend follows an MVC-inspired modular structure built on Express.js.

```

backend/
├── .env.example
├── package.json
├── server.js                # App entry (Express + Socket.io)
├── config/
│   └── db.js                # MongoDB Atlas connection
├── models/                  # Mongoose schemas
│   ├── User.js
│   ├── Listing.js
│   ├── Transaction.js
│   ├── Wallet.js
│   ├── MeterReading.js
│   ├── CarbonCredit.js
│   ├── Notification.js
│   └── Dispute.js
├── controllers/             # Request handlers / business logic
│   ├── authController.js
│   ├── listingController.js
│   ├── meterController.js
│   ├── transactionController.js
│   ├── walletController.js
│   ├── adminController.js
│   ├── matchController.js
│   ├── pricingController.js
│   ├── carbonController.js
│   ├── notificationController.js
│   ├── disputeController.js
│   └── reportController.js
├── routes/                  # API route definitions (56 endpoints)
│   ├── authRoutes.js
│   ├── listingRoutes.js
│   ├── meterRoutes.js
│   ├── transactionRoutes.js
│   ├── walletRoutes.js
│   ├── adminRoutes.js
│   ├── matchRoutes.js
│   ├── pricingRoutes.js
│   ├── carbonRoutes.js
│   ├── notificationRoutes.js
│   ├── disputeRoutes.js
│   └── reportRoutes.js
├── middleware/
│   ├── auth.js              # JWT protect + role authorize
│   └── roleCheck.js
├── services/
│   ├── matchingEngine.js    # Rank listings for buyers
│   ├── pricingEngine.js     # Suggest ₹/kWh + market snapshot
│   └── meterSimulator.js     # Background smart-meter data
└── utils/

```



```

└─ generateToken.js    # JWT helper
└─ notify.js          # Create notifications

```

## 17. API DOCUMENTATION

The backend exposes a versioned REST API under the base path `"/api/v1"`. All requests and responses use JSON. Except for the registration and login endpoints, every endpoint requires an `"Authorization: Bearer <token>"` header containing a valid JWT. Role-based middleware further restricts certain endpoints to specific roles, as noted in each endpoint's description below.

### 17.1 Authentication APIs

Total APIs: 56

Base URL: <http://localhost:5002/api>

Auth: JWT via httpOnly cookie or `Authorization: Bearer <token>`

#### 1. Health (1)

| # | Method | Endpoint                 | Access | Description / Use                                                                      |
|---|--------|--------------------------|--------|----------------------------------------------------------------------------------------|
| 1 | GET    | <code>/api/health</code> | Public | Checks if the API server is running. Used for monitoring and deployment health checks. |

#### 2. Authentication (7)

| # | Method | Endpoint                        | Access | Description / Use                                                                  |
|---|--------|---------------------------------|--------|------------------------------------------------------------------------------------|
| 2 | POST   | <code>/api/auth/register</code> | Public | Creates a new user (prosumer / consumer / admin) and demo wallet. Used for signup. |
| 3 | POST   | <code>/api/auth/login</code>    | Public | Authenticates user and returns JWT session. Used for login.                        |

|   |       |                           |           |                                                                                        |
|---|-------|---------------------------|-----------|----------------------------------------------------------------------------------------|
| 4 | POST  | /api/auth/logout          | Public    | Clears auth cookie. Used for logout.                                                   |
| 5 | GET   | /api/auth/me              | Protected | Returns current logged-in user profile. Used for session restore and navbar/dashboard. |
| 6 | PATCH | /api/auth/profile         | Protected | Updates name, phone, city, solar capacity. Used for profile settings.                  |
| 7 | POST  | /api/auth/change-password | Protected | Changes account password after verifying current password. Used for account security.  |
| 8 | POST  | /api/auth/kyc             | Protected | Submits KYC document URL for verification. Used for identity verification flow.        |

### 3. Energy Listings / Marketplace (7)

| #  | Method | Endpoint                 | Access   | Description / Use                                                                          |
|----|--------|--------------------------|----------|--------------------------------------------------------------------------------------------|
| 9  | GET    | /api/listings            | Public   | Browses active energy listings (filter by price/city/sort). Used by marketplace page.      |
| 10 | GET    | /api/listings/mine       | Prosumer | Lists all listings created by the logged-in prosumer. Used on seller dashboard.            |
| 11 | GET    | /api/listings/stats/mine | Prosumer | Returns seller listing stats by status (active/sold/cancelled). Used for seller analytics. |
| 12 | POST   | /api/listings            | Prosumer | Creates a new energy listing (kWh + price). Used to sell surplus solar energy.             |

|    |       |                          |                  |                                                                              |
|----|-------|--------------------------|------------------|------------------------------------------------------------------------------|
| 13 | GET   | /api/listings/:id        | Public           | Fetches one listing with seller details. Used for listing detail / buy page. |
| 14 | PATCH | /api/listings/:id        | Prosumer (owner) | Updates price, available kWh, or location. Used to edit an active listing.   |
| 15 | POST  | /api/listings/:id/cancel | Prosumer (owner) | Cancels an active listing. Used when seller no longer wants to sell.         |

## 4. Matching Engine (1)

| #  | Method | Endpoint   | Access    | Description / Use                                                                                                    |
|----|--------|------------|-----------|----------------------------------------------------------------------------------------------------------------------|
| 16 | POST   | /api/match | Protected | Ranks active listings for a buyer request using price, distance, and rating. Used for "best seller" recommendations. |

## 5. Dynamic Pricing (2)

| #  | Method | Endpoint             | Access    | Description / Use                                                                      |
|----|--------|----------------------|-----------|----------------------------------------------------------------------------------------|
| 17 | GET    | /api/pricing/suggest | Protected | Suggests ₹/kWh from recent trades and active listings. Used when creating a listing.   |
| 18 | GET    | /api/pricing/market  | Protected | Returns platform market snapshot and city-wise price ranges. Used for market insights. |

## 6. Smart Meter (4)

| # | Method | Endpoint | Access | Description / Use |
|---|--------|----------|--------|-------------------|
|---|--------|----------|--------|-------------------|

|    |     |                      |           |                                                                                   |
|----|-----|----------------------|-----------|-----------------------------------------------------------------------------------|
| 19 | GET | /api/meter/latest    | Protected | Latest generation/consumption reading for the user. Used on dashboards.           |
| 20 | GET | /api/meter/history   | Protected | Historical meter readings (today / week / month). Used for charts/graphs.         |
| 21 | GET | /api/meter/surplus   | Protected | Latest surplus kWh available to sell. Used before creating listings.              |
| 22 | GET | /api/meter/grid-load | Protected | Aggregated generation vs consumption (grid load). Used for grid/admin monitoring. |

## 7. Transactions (5)

| #  | Method | Endpoint                      | Access                   | Description / Use                                                                          |
|----|--------|-------------------------------|--------------------------|--------------------------------------------------------------------------------------------|
| 23 | POST   | /api/transactions/purchase    | Protected                | Buys energy from a listing; settles wallets and issues carbon credits. Core P2P trade API. |
| 24 | GET    | /api/transactions/mine        | Protected                | Lists user's buy/sell transactions. Used on transactions page.                             |
| 25 | GET    | /api/transactions/stats       | Protected                | Buy/sell volume and amount aggregates. Used for dashboard KPIs.                            |
| 26 | GET    | /api/transactions/:id         | Protected (party/admin)  | Full detail of one transaction. Used for receipt/order detail.                             |
| 27 | POST   | /api/transactions/:id/dispute | Protected (buyer/seller) | Opens a dispute on a trade. Used for conflict resolution.                                  |

## 8. Wallet (5)

| #  | Method | Endpoint             | Access    | Description / Use                                                                   |
|----|--------|----------------------|-----------|-------------------------------------------------------------------------------------|
| 28 | GET    | /api/wallet          | Protected | Full wallet with balance and history. Used on wallet page.                          |
| 29 | GET    | /api/wallet/balance  | Protected | Lightweight balance-only response. Used for quick UI badges.                        |
| 30 | GET    | /api/wallet/history  | Protected | Paginated credit/debit ledger (filter by type). Used for transaction history table. |
| 31 | POST   | /api/wallet/top-up   | Protected | Adds demo funds to wallet. Used for testing purchases.                              |
| 32 | POST   | /api/wallet/withdraw | Protected | Withdraws demo funds from wallet. Used for payout simulation.                       |

## 9. Carbon Credits (3)

| #  | Method | Endpoint                    | Access                  | Description / Use                                                                       |
|----|--------|-----------------------------|-------------------------|-----------------------------------------------------------------------------------------|
| 33 | GET    | /api/carbon/mine            | Protected               | Lists carbon credits earned by the user. Used on green impact / credits page.           |
| 34 | GET    | /api/carbon/summary         | Protected               | Totals for kWh sold, CO <sub>2</sub> saved, and credits. Used for impact summary cards. |
| 35 | GET    | /api/carbon/:id/certificate | Protected (owner/admin) | Returns certificate metadata for a credit. Used for download/share certificate.         |

## 10. Notifications (4)

| # | Method | Endpoint | Access | Description / Use |
|---|--------|----------|--------|-------------------|
|---|--------|----------|--------|-------------------|

|    |        |                             |           |                                                                      |
|----|--------|-----------------------------|-----------|----------------------------------------------------------------------|
| 36 | GET    | /api/notifications          | Protected | Lists user notifications + unread count. Used for bell icon / inbox. |
| 37 | POST   | /api/notifications/read-all | Protected | Marks all notifications as read. Used for “mark all read”.           |
| 38 | PATCH  | /api/notifications/:id/read | Protected | Marks one notification as read. Used when opening a notification.    |
| 39 | DELETE | /api/notifications/:id      | Protected | Deletes a notification. Used to clear inbox items.                   |

## 11. Disputes (2)

| #  | Method | Endpoint           | Access                  | Description / Use                                                           |
|----|--------|--------------------|-------------------------|-----------------------------------------------------------------------------|
| 40 | GET    | /api/disputes/mine | Protected               | Lists disputes raised by or against the user. Used on user dispute page.    |
| 41 | GET    | /api/disputes/:id  | Protected (party/admin) | Fetches one dispute with transaction details. Used for dispute detail view. |

## 12. Reports (2)

| #  | Method | Endpoint             | Access    | Description / Use                                                                                |
|----|--------|----------------------|-----------|--------------------------------------------------------------------------------------------------|
| 42 | GET    | /api/reports/impact  | Protected | Personal impact report (energy, money, CO <sub>2</sub> , listings). Used for user report/export. |
| 43 | GET    | /api/reports/monthly | Protected | Month-wise buy/sell series. Used for monthly analytics charts.                                   |

## 13. Admin (13)

| #  | Method | Endpoint                            | Access | Description / Use                                                             |
|----|--------|-------------------------------------|--------|-------------------------------------------------------------------------------|
| 44 | GET    | /api/admin/overview                 | Admin  | Platform KPIs: users, listings, energy sold, volume. Used on admin dashboard. |
| 45 | GET    | /api/admin/users                    | Admin  | Lists all users. Used for user management.                                    |
| 46 | PATCH  | /api/admin/users/:id/block          | Admin  | Blocks/unblocks a user. Used for moderation.                                  |
| 47 | PATCH  | /api/admin/users/:id/verify         | Admin  | Marks a user as KYC-verified. Used after document review.                     |
| 48 | DELETE | /api/admin/users/:id                | Admin  | Deletes a user account. Used for account removal.                             |
| 49 | GET    | /api/admin/listings                 | Admin  | Lists all marketplace listings. Used for listing moderation.                  |
| 50 | DELETE | /api/admin/listings/:id             | Admin  | Force-deletes a listing. Used to remove invalid listings.                     |
| 51 | GET    | /api/admin/transactions             | Admin  | Lists all platform transactions. Used for settlement audit.                   |
| 52 | PATCH  | /api/admin/transactions/:id/resolve | Admin  | Resolves a disputed transaction status. Used for quick dispute close.         |
| 53 | GET    | /api/admin/meters                   | Admin  | Recent smart-meter readings across users. Used for grid/meter monitoring.     |
| 54 | GET    | /api/admin/disputes                 | Admin  | Full dispute queue with parties and reasons. Used for dispute management.     |

|    |       |                         |       |                                                                                            |
|----|-------|-------------------------|-------|--------------------------------------------------------------------------------------------|
| 55 | PATCH | /api/admin/disputes/:id | Admin | Updates dispute status (resolved/rejected/under review). Used for formal dispute handling. |
| 56 | GET   | /api/admin/carbon       | Admin | Lists all issued carbon credits. Used for environmental audit.                             |

## Summary for report

| Module                 | API Count |
|------------------------|-----------|
| Health                 | 1         |
| Authentication         | 7         |
| Listings / Marketplace | 7         |
| Matching Engine        | 1         |
| Dynamic Pricing        | 2         |
| Smart Meter            | 4         |
| Transactions           | 5         |
| Wallet                 | 5         |
| Carbon Credits         | 3         |
| Notifications          | 4         |
| Disputes               | 2         |
| Reports                | 2         |
| Admin                  | 13        |
| Total                  | 56        |

Roles: `prosumer` (sell energy), `consumer` (buy energy), `admin` (platform control)

Tech stack: Express.js REST APIs + MongoDB Atlas (Mongoose) + JWT auth



## 24. REAL-TIME APPLICATIONS

Several aspects of the system are designed to reflect real-time or near-real-time behaviour, which is central to demonstrating the value proposition of a live peer-to-peer energy marketplace as opposed to a static, monthly-billed utility model. The smart-meter simulation service continuously generates new readings on a fixed interval, immediately updating each account's available surplus and, by extension, the maximum quantity that account is eligible to list for sale. Dashboard screens for all three roles are designed to refresh at short, regular intervals (or via lightweight polling of the dashboard summary and grid-load endpoints), so that generation, consumption, surplus, and grid-load charts reflect current, not merely historical, data. The marketplace listing feed similarly reflects newly created, partially filled, or cancelled listings promptly, so that Consumers always see an accurate, current picture of available supply and Prosumers see immediate confirmation when their energy is purchased. While the current implementation uses efficient polling rather than a persistent WebSocket connection, the architecture is compatible with a future upgrade to Socket.IO or native WebSockets for push-based real-time updates, as discussed in the Future Enhancements section.

## 25. ADVANTAGES

- Gives solar Prosumers direct control over the price at which they sell surplus energy, rather than accepting a fixed utility buy-back rate.
- Provides Consumers a transparent, competitively priced alternative source of renewable energy from nearby producers.
- Offers real-time visibility into generation, consumption, surplus, earnings, and spending, rather than a delayed monthly bill.
- Demonstrates a scalable, modular MERN stack architecture that can be extended with real hardware and payment integrations.
- Maintains an immutable, auditable transaction ledger suitable as a foundation for future blockchain-based settlement.
- Role-based access control cleanly separates Prosumer, Consumer, and Grid Administrator concerns, simplifying both security and UI design.
- The greedy best-price matching algorithm is simple to understand, test, and reason about while still producing economically sensible outcomes.
- Provides the Grid Administrator with an aggregate, system-wide view of grid load and marketplace health that is difficult to obtain from legacy utility billing systems.

## 26. LIMITATIONS

- The system uses simulated smart-meter readings rather than data from real physical meters or IoT devices.
- All monetary values represent a virtual, in-application balance rather than real currency; no real payment gateway is integrated.
- The matching engine uses a simple greedy best-price algorithm and does not account for network/proximity constraints, transmission losses, or multi-attribute auction mechanisms used in some advanced energy markets.
- The current implementation does not integrate blockchain or distributed-ledger technology, relying instead on a centralized MongoDB database for the transaction ledger.
- Real-time updates are implemented via polling rather than persistent WebSocket connections, which introduces a small delay compared to true push-based updates.
- The system does not currently model detailed grid topology, transformer capacity limits, or physical transmission constraints between specific Prosumer-Consumer pairs.
- Regulatory, tariff, and legal compliance considerations relevant to an actual commercial energy-trading platform are outside the scope of this academic project.

## 27. FUTURE ENHANCEMENTS

Several enhancements could extend this project from an academic simulation toward a more production-ready system. Integration with real IoT-based smart meters (for example, using MQTT or a similar lightweight messaging protocol) would replace the simulation engine with genuine generation and consumption data. A blockchain-based settlement layer, using a permissioned ledger or smart contracts, could provide stronger tamper-resistance and third-party auditability guarantees for the transaction ledger than a centralized database alone. Real-time push-based updates using WebSockets (via Socket.IO) would replace the current polling mechanism, reducing latency and server load. Machine-learning-based demand and generation forecasting could allow the system to suggest optimal listing prices to Prosumers and proactively alert Consumers to upcoming supply gluts or shortages. Integration with a real payment gateway (for example, a UPI or card-based processor) would allow the virtual wallet balance to be converted into real settlement. A more sophisticated matching engine incorporating physical proximity, grid topology, and transmission-loss modelling would improve the realism of the marketplace. Finally, a mobile application (React Native) could complement the web application, and a notification system (email/SMS/push) could alert users to completed trades, low surplus, or unusual grid conditions flagged by the administrator.

## 28. CONCLUSION

This project has successfully designed and implemented a full-stack MERN application simulating a peer-to-peer solar energy trading marketplace, addressing the core limitations of the traditional, one-directional, utility-controlled feed-in-tariff model. Through a smart-meter simulation engine, an energy marketplace module, an automated buyer-seller matching engine, an immutable transaction ledger, and role-specific real-time dashboards, the system demonstrates that a decentralized, transparent, and engaging local energy trading experience can be built using widely available open-source web technologies.

The project required careful requirement analysis, thoughtful database and API design, and disciplined attention to security and testing practices, all of which have been documented in detail throughout this report. While the current implementation remains an academic simulation rather than a certified, grid-connected commercial product, it establishes a sound and extensible architectural foundation. The identified future enhancements — including real hardware integration, blockchain-based settlement, push-based real-time updates, and demand forecasting — outline a credible path from this prototype toward a more complete, production-grade peer-to-peer energy trading platform.

In conclusion, this project has met its stated objectives, providing hands-on experience with the complete MERN stack development lifecycle, from requirement analysis and system design through implementation, security hardening, and structured testing, while engaging meaningfully with an important and growing real-world problem in the renewable energy domain.

## 29. REFERENCES

- MongoDB, Inc. MongoDB Documentation. Available at: <https://www.mongodb.com/docs/>
- Mongoose. Mongoose ODM Documentation. Available at: <https://mongoosejs.com/docs/>
- OpenJS Foundation. Express.js Documentation. Available at: <https://expressjs.com/>
- Meta Platforms, Inc. React Documentation. Available at: <https://react.dev/>
- Vite. Vite Documentation. Available at: <https://vitejs.dev/>
- Tailwind Labs. Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
- Recharts. Recharts Documentation. Available at: <https://recharts.org/>
- Auth0 / jwt.io. Introduction to JSON Web Tokens. Available at: <https://jwt.io/introduction>
- Postman, Inc. Postman Learning Center. Available at: <https://learning.postman.com/>
- Node.js Foundation. Node.js Documentation. Available at: <https://nodejs.org/docs/>
- IEEE Smart Grid. Transactive Energy Systems: Concepts and Applications. IEEE Smart Grid Publications.
- International Renewable Energy Agency (IRENA). Peer-to-Peer Electricity Trading: Innovation Landscape Brief. IRENA Publications.
- OWASP Foundation. OWASP Top Ten Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/>
- bcrypt.js. bcrypt Password Hashing Library Documentation. Available at: <https://www.npmjs.com/package/bcryptjs>